In [1]:
```python
import numpy as np

def train_hebb(X, y):
    # Initialize weights and bias
    weights = np.zeros(X.shape[1], dtype=float)
    bias = 0.0
    # Hebbian update: w = w + x*y; bias = bias + y
    for inputs, target in zip(X, y):
        weights += inputs * target
        bias += target
    return weights, bias

def predict_hebb(X, weights, bias):
    net = np.dot(X, weights) + bias
    return np.where(net >= 0, 1, -1)

def show_gate(name, X, y):
    weights, bias = train_hebb(X, y)
    predictions = predict_hebb(X, weights, bias)
    print(f"\n{name} gate")
    print("Weights:", weights, "Bias:", bias)
    print("Input  Target  Predicted")
    for x, target, pred in zip(X, y, predictions):
        print(f"{x}    {target:>3}      {pred:>3}")
    print("Correct:", np.sum(predictions == y), "/", len(y))

# Bipolar input patterns for two-input gates
X2 = np.array([[-1, -1], [-1, 1], [1, -1], [1, 1]])

# Bipolar AND: output +1 only when both inputs are +1
y_and = np.array([-1, -1, -1, 1])
# Bipolar OR: output +1 when at least one input is +1
y_or = np.array([-1, 1, 1, 1])

show_gate("AND", X2, y_and)
show_gate("OR", X2, y_or)

# NOT gate has one input
X_not = np.array([[-1], [1]])
y_not = np.array([1, -1])
show_gate("NOT", X_not, y_not)
```

```
AND gate
Weights: [2. 2.] Bias: -2.0
Input  Target  Predicted
[-1 -1]     -1       -1
[-1  1]     -1       -1
[ 1 -1]     -1       -1
[1 1]      1        1
Correct: 4 / 4

OR gate
Weights: [2. 2.] Bias: 2.0
Input  Target  Predicted
[-1 -1]     -1       -1
[-1  1]      1        1
[ 1 -1]      1        1
[1 1]      1        1
Correct: 4 / 4

NOT gate
Weights: [-2.] Bias: 0.0
Input  Target  Predicted
[-1]      1        1
[1]     -1       -1
Correct: 2 / 2
```